



Module 03 – Exercise Class

PANDAS - DATA ANALYSIS

Nguyen Quoc Thai

Objectives

Introduction

- ❖ Pandas
- ❖ Pandas Data Structures
- ❖ Creating DataFrames
- ❖ Selection
- ❖ Dropping
- ❖ Sort & Rank
- ❖ Retrieving Series / DataFrame Information
- ❖ Applying Functions
- ❖ Data Alignment

Application / Exercise

- ❖ IMDB-Dataset
- ❖ Open Power System Germany Daily Dataset

Advanced

- ❖ Polars
- ❖ Compare: Pandas and Polars



Outline

SECTION 1

Introduction

SECTION 2

Applications

SECTION 3

Polars



Introduction



Pandas

The Pandas library is built on NumPy and provides easy-to-use data structures and data analysis tools for the Python



Introduction



Pandas

The Pandas library is built on NumPy and provides easy-to-use data structures and data analysis tools for the Python



```
!pip install pandas

import pandas as pd

pd.__version__
>>> 2.2.2
```

Document

<https://pandas.pydata.org/docs/>

- Getting Started
- User Guide
- API Reference
- Developer Guide

Introduction



Pandas Data Structures

Series

A one-dimensional labeled array capable of holding any data type

Index	Data
0	3
1	-5
2	7
3	4

```
1 s = pd.Series([3, -5, 7, 4])
2 s
✓ 0.0s

0    3
1   -5
2    7
3    4
dtype: int64
```

Introduction



Pandas Data Structures

Series

A one-dimensional labeled array capable of holding any data type

Index	Data
a	3
b	-5
c	7
d	4

```
1 s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
```

```
2 s
```

```
✓ 0.0s
```

```
a    3
```

```
b   -5
```

```
c    7
```

```
d    4
```

```
dtype: int64
```

Introduction



Pandas Data Structures

DataFrame

A two-dimensional labeled data structure with columns of potentially different types

Index	Columns	
	Name	Country
0	Anh	Brazil
1	Thai	Vietnam
2	Hoa	Vietnam

```
1 data = {  
2     'Name': ['Anh', 'Thai', 'Hoa'],  
3     'Country': ['Brazil', 'Vietnam', 'Vietnam']  
4 }  
5 df = pd.DataFrame(data, columns=['Name', 'Country'])  
6 df  
✓ 0.0s
```

	Name	Country
0	Anh	Brazil
1	Thai	Vietnam
2	Hoa	Vietnam

Introduction



Basic Information

Summary data

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0



```
len(df)
df.shape
df.index
df.columns
df.info()
df.count()
df.describe()
```

	Grade
count	3.0
mean	8.0
std	1.0
min	7.0
25%	7.5
50%	8.0
75%	8.5
max	9.0



Selection

NumPy Array

Index	Data	
a	3	1 s['b'] ✓ 0.0s
b	-5	-5
c	7	
d	4	

Index	Columns	
	Name	Country
0	Anh	Brazil
1	Thai	Vietnam
2	Hoa	Vietnam

1	df[1:]	
✓	0.0s	
	Name	Country
1	Thai	Vietnam
2	Hoa	Vietnam



Selection

By Position/Label

Select single value by row & column

Index

Columns

0

Anh

Brazil

1

Thai

Vietnam

2

Hoa

Vietnam

```
1 df.iloc[[0],[0]]
```

✓ 0.0s

Name

0 Anh

```
1 df.iloc[0,0]
```

✓ 0.0s

'Anh'

```
1 df.iat[1, 2]
```

✓ 0.0s

8.0

```
1 df.at[2, 'Name']
```

✓ 0.0s

'Hoa'



Selection

By Position

Select values by row & column

Index

Columns

0

Anh

Brazil

1

Thai

Vietnam

2

Hoa

Vietnam

```
1 df.iloc[0:2]
```

✓ 0.0s

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0

```
1 df.iloc[:, 1]
```

✓ 0.0s

0 Brazil

1 Vietnam

2 Vietnam

Name: Country, dtype: object

```
1 df.iloc[:, [0, 1]]
```

✓ 0.0s

	Name	Country
0	Anh	Brazil
1	Thai	Vietnam
2	Hoa	Vietnam



Selection

By Position/Label

Select values by row & column labels

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.loc[0:2]
```

✓ 0.0s

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.loc[:,:]
```

✓ 0.0s

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0



Selection

By Position/Label

Select values by row & column labels

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.loc[0,['Name','Country']]
```

```
Name      Anh
Country    Brazil
Name: 0, dtype: object
```

```
1 df.loc[:, 'Name':'Country']
```

✓ 0.0s

```
   Name Country
0  Anh  Brazil
1  Thai Vietnam
2  Hoa  Vietnam
```



Introduction



Selection

Subset Observations - Rows

Index

Columns

0

Name

Country

Anh

Brazil

1

Thai

Vietnam

2

Hoa

Vietnam

Thai

Vietnam

Hoa

Vietnam

Introduction



Selection

Subset Observations - Rows

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.head(n=2)
```

✓ 0.0s

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0

```
1 df.sample(frac=0.5)
```

✓ 0.0s

	Name	Country	Grade
0	Anh	Brazil	7.0
2	Hoa	Vietnam	9.0

```
1 df.tail(n=2)
```

✓ 0.0s

	Name	Country	Grade
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.sample(n=2)
```

✓ 0.0s

	Name	Country	Grade
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0



Selection

Subset Observations - Rows

Index	Columns		
	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.nlargest(n=2, columns='Grade')
✓ 0.0s
```

	Name	Country	Grade
2	Hoa	Vietnam	9.0
1	Thai	Vietnam	8.0

```
1 df.nsmallest(n=2, columns='Grade')
✓ 0.0s
```

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0

```
1 df[df.Grade > 7.0]
✓ 0.0s
```

	Name	Country	Grade
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

Introduction



Selection

Subset Variables - Columns

Index

Columns

0

Anh

Brazil

1

Thai

Vietnam

2

Hoa

Vietnam



Name

Anh

Thai

Hoa



Selection

Subset Variables - Columns

Index	Columns		
	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df[['Name']]
✓ 0.0s
```

Name	
0	Anh
1	Thai
2	Hoa

```
1 df.Grade
✓ 0.0s
```

0	7.0
1	8.0
2	9.0

Name: Grade, dtype: float64

Introduction



Dropping

Drop values from rows or columns

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.drop([0, 1])
```

✓ 0.0s

	Name	Country	Grade
2	Hoa	Vietnam	9.0

```
1 df.drop('Grade', axis=1)
```

✓ 0.0s

	Name	Country
0	Anh	Brazil
1	Thai	Vietnam
2	Hoa	Vietnam



Sorting

Sort by labels along an axis or values along axis

Index	Columns		
	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.sort_index(axis=1)
```

✓ 0.0s

	Country	Grade	Name
0	Brazil	7.0	Anh
1	Vietnam	8.0	Thai
2	Vietnam	9.0	Hoa

```
1 df.sort_values('Name')
```

✓ 0.0s

	Name	Country	Grade
0	Anh	Brazil	7.0
2	Hoa	Vietnam	9.0
1	Thai	Vietnam	8.0

Introduction



Applying Functions

Apply function to each object

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 f = lambda x: x*2
2 df['Grade'].apply(f)
```

✓ 0.0s

0 14.0

1 16.0

2 18.0

Name: Grade, dtype: float64

```
1 f = lambda x: [1, 2]
2 df.apply(f)
```

✓ 0.0s

	Name	Country	Grade
0	1	1	1
1	2	2	2



Applying Functions

Apply function to each object

Index	Columns		
	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 f = lambda x: [1, 2, 3]
2 df.apply(f, axis=1)

✓ 0.0s
```

```
0    [1, 2, 3]
1    [1, 2, 3]
2    [1, 2, 3]
dtype: object
```

```
1 f = lambda x: [1, 2, 3]
2 df.apply(f, axis=1, result_type='expand')

✓ 0.0s
```

```
    0  1  2
0  1  2  3
1  1  2  3
2  1  2  3
```

Introduction



Change layout

Reindexing - Renaming

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.rename(columns={'Grade': 'Score'})
```

✓ 0.0s

	Name	Country	Score
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

Introduction



Change layout

Reindexing - Renaming

Index	Columns		
	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.set_index('Grade')
```

✓ 0.0s

	Name	Country
Grade		
7.0	Anh	Brazil
8.0	Thai	Vietnam
9.0	Hoa	Vietnam

```
1 df.set_index('Grade').reset_index()
```

✓ 0.0s

	Grade	Name	Country
0	7.0	Anh	Brazil
1	8.0	Thai	Vietnam
2	9.0	Hoa	Vietnam



Replace – Group Data

Return a GroupBy Object

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df.replace({7.0:8.0})
```

✓ 0.0s

	Name	Country	Grade
0	Anh	Brazil	8.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 newdf = df.replace({7.0:8.0})
```

```
2 newdf.groupby(by='Grade').min()
```

✓ 0.0s

	Name	Country
Grade		
8.0	Anh	Brazil
9.0	Hoa	Vietnam

Introduction



Rolling

Return a **Rolling object** allowing summary functions to be applied to **window size**

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df['Grade'].rolling(3)
```

✓ 0.0s

Rolling [window=3,center=False,axis=0,method=single]

```
1 df['Grade'].rolling(3).sum()
```

✓ 0.0s

0 NaN

1 NaN

2 24.0

Name: Grade, dtype: float64

Introduction



Rolling

Return a **Rolling object** allowing summary functions to be applied to **window size**

Index

Columns

	Name	Country	Grade
0	Anh	Brazil	7.0
1	Thai	Vietnam	8.0
2	Hoa	Vietnam	9.0

```
1 df['Grade'].rolling(3, center=True)
```

✓ 0.0s

Rolling [window=3,center=True,axis=0,method=single]

```
1 df['Grade'].rolling(3, center=True).sum()
```

✓ 0.0s

```
0    NaN
1    24.0
2    NaN
Name: Grade, dtype: float64
```

Outline

SECTION 1

Introduction

IMDB Movie Dataset

SECTION 2

Applications

SECTION 3

Polars



IMDB Movie Dataset



IMDB Movie Dataset

Problem Statement: Given IMDB Movie dataset, do some analysis using Pandas including: View, Describe, Info, Data Selection, GroupBy, Sorting, Fill NaN, Data Processing.

1 data

✓ 0.0s

Python

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117

IMDB Movie Dataset



Load Dataset

```
1 dataset_path = 'IMDB-Movie-Data.csv'
2
3 # Read data from .csv file
4 data = pd.read_csv(dataset_path)
5
6 # Read data with specified explicit index.
7 # We will use this later in our analysis
8 data_indexed = pd.read_csv(dataset_path, index_col="Title")
```

1 data_indexed

✓ 0.0s

Python

	Rank	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rati
Title								
Guardians of the Galaxy	1	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8

IMDB Movie Dataset



Data Information

```
1 data.describe()
```

✓ 0.0s

	Rank	Year	Runtime (Minutes)	Rating
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	2012.783000	113.172000	6.723200
std	288.819436	3.205962	18.810908	0.945429
min	1.000000	2006.000000	66.000000	1.900000
25%	250.750000	2010.000000	100.000000	6.200000
50%	500.500000	2014.000000	111.000000	6.800000
75%	750.250000	2016.000000	123.000000	7.400000
max	1000.000000	2016.000000	191.000000	9.000000

```
1 data.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Rank	1000 non-null	int64
1	Title	1000 non-null	object
2	Genre	1000 non-null	object
3	Description	1000 non-null	object
4	Director	1000 non-null	object
5	Actors	1000 non-null	object
6	Year	1000 non-null	int64
7	Runtime (Minutes)	1000 non-null	int64
8	Rating	1000 non-null	float64
9	Votes	1000 non-null	int64
10	Revenue (Millions)	872 non-null	float64
11	Metascore	936 non-null	float64

```
dtypes: float64(3), int64(4), object(5)
```

```
memory usage: 93.9+ KB
```


IMDB Movie Dataset



Data Selection

```
1 genre = data['Genre']
2 genre
```

✓ 0.0s

```
0    Action,Adventure,Sci-Fi
1    Adventure,Mystery,Sci-Fi
2           Horror,Thriller
3    Animation,Comedy,Family
4    Action,Adventure,Fantasy
...
995    Crime,Drama,Mystery
996           Horror
997    Drama,Music,Romance
998    Adventure,Comedy
999    Comedy,Family,Fantasy
Name: Genre, Length: 1000, dtype: object
```

```
1 columns = data[['Title','Genre','Actors','Director','Rating']]
2 columns
```

✓ 0.0s

	Title	Genre	
0	Guardians of the Galaxy	Action,Adventure,Sci-Fi	Chris Pratt, Vin
1	Prometheus	Adventure,Mystery,Sci-Fi	Noomi Rapace, Greer
2	Split	Horror,Thriller	James McAvoy, A Hal
3	Sing	Animation,Comedy,Family	Matthew Mc Withersp

IMDB Movie Dataset



Data Selection

```
1 data.set_index('Title').loc[['Suicide Squad']]
```

✓ 0.0s

	Rank	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes
Title									
Suicide Squad	5	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727

IMDB Movie Dataset



Data Selection

```
1 data.iloc[10:15][['Title', 'Rating', 'Revenue (Millions)']]
```

✓ 0.0s

	Title	Rating	Revenue (Millions)
10	Fantastic Beasts and Where to Find Them	7.5	234.02
11	Hidden Figures	7.8	169.27
12	Rogue One	7.9	532.17
13	Moana	7.7	248.75
14	Colossal	6.4	2.87

IMDB Movie Dataset

! Data Selection – Conditional Filtering

Select statement: Select all movies publish **between 2010 and 2015**, have **rating less than 6.0** and **revenue over 95'th percentile** of dataset.

```
1 data[((data['Year'] >= 2010) & (data['Year'] <= 2015))
2 |      & (data['Rating'] < 6.0)
3 |      & (data['Revenue (Millions)'] > data['Revenue (Millions)'].quantile(0.95))]
```

✓ 0.0s

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)
941	942	The Twilight Saga: Eclipse	Adventure,Drama,Fantasy	As a string of mysterious killings grips Seatt...	David Slade	Kristen Stewart, Robert Pattinson, Taylor Laut...	2010	124

IMDB Movie Dataset



GroupBy Operations

```
1 data.groupby('Director')[['Rating']].mean().head()
```

✓ 0.0s

	Rating
Director	
Aamir Khan	8.5
Abdellatif Kechiche	7.8
Adam Leon	6.5
Adam McKay	7.0
Adam Shankman	6.3

```
1 data[['Director', 'Rating']]
```

✓ 0.0s

	Director	Rating
0	James Gunn	8.1
1	Ridley Scott	7.0
2	M. Night Shyamalan	7.3
3	Christophe Lourdelet	7.2
4	David Ayer	6.2
...	...	...
995	Billy Ray	6.2
996	Eli Roth	5.5
997	Jon M. Chu	6.2
998	Scot Armstrong	5.6
999	Barry Sonnenfeld	5.3

1000 rows × 2 columns

IMDB Movie Dataset



Sorting

```
1 x = data.groupby('Director')[['Rating']].mean()  
2 x.sort_values(['Rating'], ascending=False).head()
```

✓ 0.0s

	Rating
Director	
Nitesh Tiwari	8.80
Christopher Nolan	8.68
Olivier Nakache	8.60
Makoto Shinkai	8.60
Aamir Khan	8.50

IMDB Movie Dataset



View Missing Values

```
1 # To check null values row-wise
2 data.isnull().sum()
```

✓ 0.0s

Rank	0
Title	0
Genre	0
Description	0
Director	0
Actors	0
Year	0
Runtime (Minutes)	0
Rating	0
Votes	0
Revenue (Millions)	128
Metascore	64
dtype:	int64

```
1 data[data.isnull().any(axis=1)]
```

✓ 0.0s

Rank		Title	Genre	Description
7	8	Mindhorn	Comedy	A has-been actor best known for playing the ti...
22	23	Hounds of Love	Crime,Drama,Horror	A cold-blooded predatory couple while cruising...



Deal With Missing Values – Deleting

```
1 data.dropna().shape
```

✓ 0.0s

(838, 12)

```
1 data.dropna()
```

✓ 0.0s

	Rank	Title	Genre	Description	Director
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott

IMDB Movie Dataset



Deal With Missing Values – Filling

```
1 data.fillna(300).shape
```

✓ 0.0s

(1000, 12)

```
1 data.fillna(300)
```

✓ 0.0s

	Rank	Title	Genre	Description	Director
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott

IMDB Movie Dataset



Classify Movie Based On Ratings

```
1 # Classify movies based on ratings
2 def rating_group(rating):
3     if rating >= 7.5:
4         return 'Good'
5     elif rating >= 6.0:
6         return 'Average'
7     else:
8         return 'Bad'
9
10 # Lets apply this function on our movies data
11 # creating a new variable in the dataset to hold the rating category
12 data['Rating_category'] = data['Rating'].apply(rating_group)
13
14 data[['Title', 'Director', 'Rating', 'Rating_category']].head(5)
```

✓ 0.0s

	Title	Director	Rating	Rating_category
0	Guardians of the Galaxy	James Gunn	8.1	Good
1	Prometheus	Ridley Scott	7.0	Average
2	Split	M. Night Shyamalan	7.3	Average
3	Sing	Christophe Lourdelet	7.2	Average
4	Suicide Squad	David Ayer	6.2	Average

Outline

SECTION 1

Introduction

Open Power System Data

SECTION 2

Applications

SECTION 3

Polars



Open Power System Data



Open Power System Data – OPSD Time Seri Dataset

```
1 dataset_path = './opds_germany_daily.csv'
2
3 # Read data from .csv file
4 opsd_daily = pd.read_csv(dataset_path)
5
6 print(opsd_daily.shape)
7 print(opsd_daily.dtypes)
8 opsd_daily.head(3)
```

✓ 0.0s

(4383, 5)

Date object

Consumption float64

Wind float64

Solar float64

Wind+Solar float64

dtype: object

	Date	Consumption	Wind	Solar	Wind+Solar
0	2006-01-01	1069.184	NaN	NaN	NaN
1	2006-01-02	1380.521	NaN	NaN	NaN
2	2006-01-03	1442.533	NaN	NaN	NaN

Open Power System Data



Set Index: 'Date' – A unique value

```
1 opsd_daily = opsd_daily.set_index('Date')  
2 opsd_daily.head(3)
```

✓ 0.0s

	Consumption	Wind	Solar	Wind+Solar
Date				
2006-01-01	1069.184	NaN	NaN	NaN
2006-01-02	1380.521	NaN	NaN	NaN
2006-01-03	1442.533	NaN	NaN	NaN

Open Power System Data



Extract Date Format

```
1 opsd_daily = pd.read_csv(dataset_path, index_col=0, parse_dates=True)
2
3 opsd_daily['Year'] = opsd_daily.index.year
4 opsd_daily['Month'] = opsd_daily.index.month
5 opsd_daily['Weekday Name'] = opsd_daily.index.day_name()
6
7 opsd_daily.sample(5, random_state=0)
```

✓ 0.0s

	Consumption	Wind	Solar	Wind+Solar	Year	Month	Weekday Name
Date							
2008-08-23	1152.011	NaN	NaN	NaN	2008	8	Saturday
2013-08-08	1291.984	79.666	93.371	173.037	2013	8	Thursday
2009-08-27	1281.057	NaN	NaN	NaN	2009	8	Thursday
2015-10-02	1391.050	81.229	160.641	241.870	2015	10	Friday
2009-06-02	1201.522	NaN	NaN	NaN	2009	6	Tuesday

Open Power System Data



Time-based Indexing

```
1 opsd_daily.loc['2014-01-20':'2014-01-22']
```

✓ 0.0s

	Consumption	Wind	Solar	Wind+Solar	Year	Month	Weekday	Name
Date								
2014-01-20	1590.687	78.647	6.371	85.018	2014	1	Monday	
2014-01-21	1624.806	15.643	5.835	21.478	2014	1	Tuesday	
2014-01-22	1625.155	60.259	11.992	72.251	2014	1	Wednesday	

```
1 new_opsd_df = opsd_daily.loc['2012-02']
```

```
2 new_opsd_df.head()
```

✓ 0.0s

	Consumption	Wind	Solar	Wind+Solar	Year	Month	Weekday	Name
Date								
2012-02-01	1511.866	199.607	43.502	243.109	2012	2	Wednesday	
2012-02-02	1563.407	73.469	44.675	118.144	2012	2	Thursday	
2012-02-03	1563.631	36.352	46.510	82.862	2012	2	Friday	
2012-02-04	1372.614	20.551	45.225	65.776	2012	2	Saturday	
2012-02-05	1279.432	55.522	54.572	110.094	2012	2	Sunday	

Open Power System Data

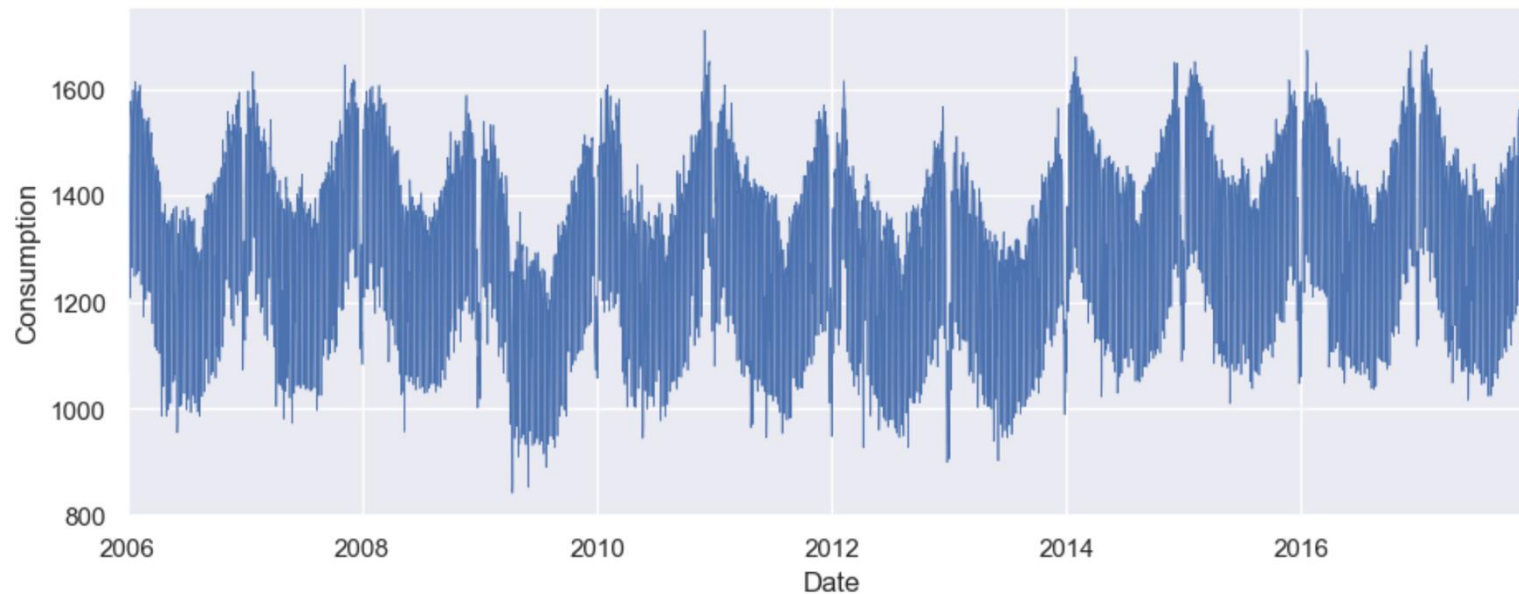


Visualizing Time Series Data

```
1 sns.set(rc={'figure.figsize':(11, 4)})  
2 col_to_plot = 'Consumption'  
3 opsd_daily[col_to_plot].plot(linewidth=0.5)  
4 plt.ylabel(col_to_plot)  
5 plt.show()
```

✓ 0.3s

Python



Open Power System Data

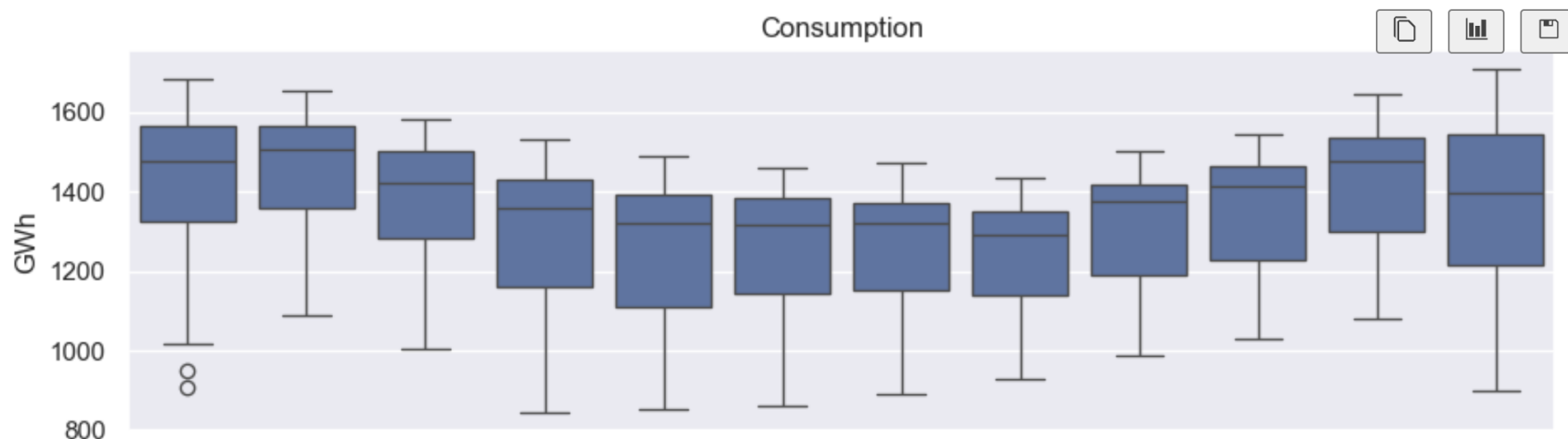


Seasonality

```
1 fig, axes = plt.subplots(3, 1, figsize=(11, 10), sharex=True)
2 for name, ax in zip(['Consumption', 'Solar', 'Wind'], axes):
3     sns.boxplot(data=opsd_daily, x='Month', y=name, ax=ax)
4     ax.set_ylabel('GWh')
5     ax.set_title(name)
6     # Remove the automatic x-axis label from all but the bottom subplot
7     if ax != axes[-1]:
8         ax.set_xlabel('')
```

✓ 0.6s

Python



Open Power System Data



Frequencies

```
1 pd.date_range('1998-03-10', '1998-03-15', freq='D')
```

✓ 0.0s

```
DatetimeIndex(['1998-03-10', '1998-03-11', '1998-03-12', '1998-03-13',  
              '1998-03-14', '1998-03-15'],  
              dtype='datetime64[ns]', freq='D')
```

```
1 pd.date_range('2004-09-20', periods=8, freq='h')
```

✓ 0.0s

```
DatetimeIndex(['2004-09-20 00:00:00', '2004-09-20 01:00:00',  
              '2004-09-20 02:00:00', '2004-09-20 03:00:00',  
              '2004-09-20 04:00:00', '2004-09-20 05:00:00',  
              '2004-09-20 06:00:00', '2004-09-20 07:00:00'],  
              dtype='datetime64[ns]', freq='h')
```

Open Power System Data



Frequencies

```
1 # To select an arbitrary sequence of date/time values from a pandas time series,  
2 # we need to use a DatetimeIndex, rather than simply a list of date/time strings  
3 times_sample = pd.to_datetime(['2013-02-03', '2013-02-06', '2013-02-08'])  
4 # Select the specified dates and just the Consumption column  
5 consum_sample = opsd_daily.loc[times_sample, ['Consumption']].copy()  
6 consum_sample
```

✓ 0.0s

	Consumption
2013-02-03	1109.639
2013-02-06	1451.449
2013-02-08	1433.098

Open Power System Data



Frequencies

```
1 # Convert the data to daily frequency, without filling any missings
2 consum_freq = consum_sample.asfreq('D')
3 # Create a column with missings forward filled
4 consum_freq['Consumption - Forward Fill'] = consum_sample.asfreq('D', method='ffill')
5 consum_freq
```

✓ 0.0s

	Consumption	Consumption - Forward Fill
2013-02-03	1109.639	1109.639
2013-02-04	NaN	1109.639
2013-02-05	NaN	1109.639
2013-02-06	1451.449	1451.449
2013-02-07	NaN	1451.449
2013-02-08	1433.098	1433.098

Open Power System Data



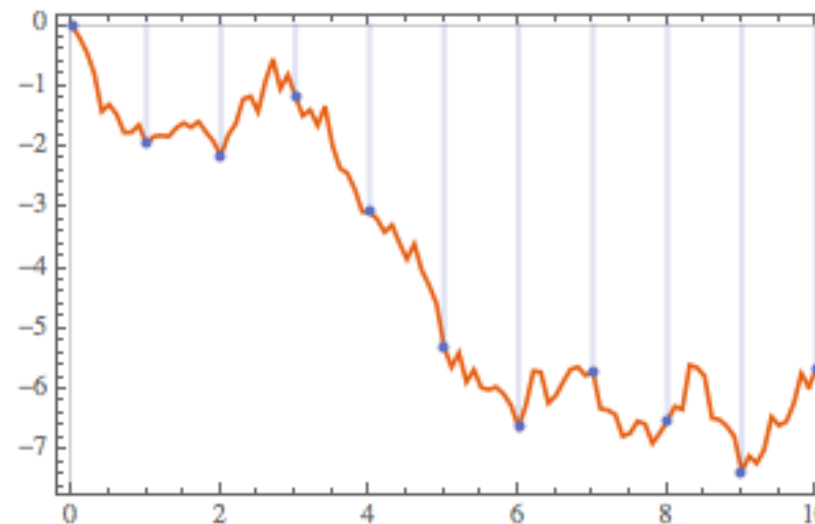
Resampling

15/03	16/03	17/03	18/03	19/03	20/03	21/03
1	3	1	2	4	1	

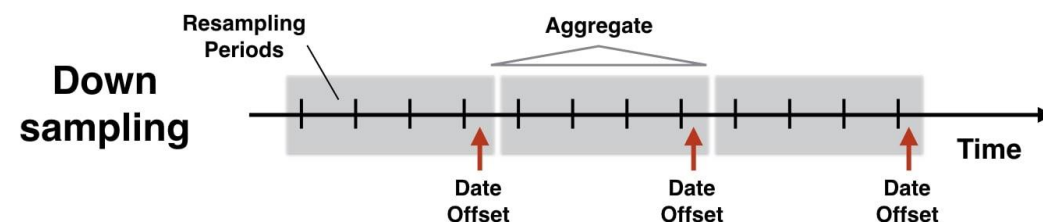
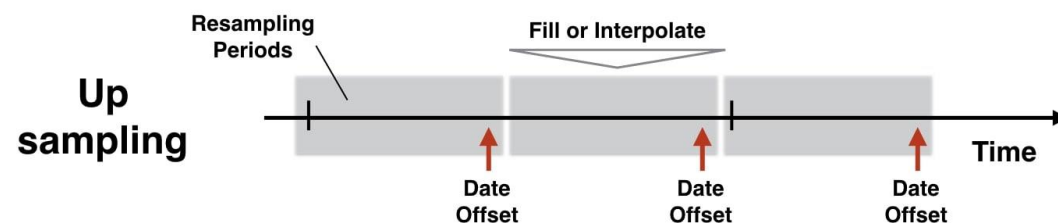
Time series by 1 day

15/03	17/03	19/03	21/03	23/03	25/03	27/03
2	1.5	2.5	...	...	...	...

Time series by 2 days



— original time series
• resampled time series



Open Power System Data



Resampling

```
1 data_columns = ['Consumption', 'Wind', 'Solar', 'Wind+Solar']
2 opsd_weekly_mean = opsd_daily[data_columns].resample('W').mean()
3 opsd_weekly_mean.head(3)
```

✓ 0.0s

	Consumption	Wind	Solar	Wind+Solar
Date				
2006-01-01	1069.184000	NaN	NaN	NaN
2006-01-08	1381.300143	NaN	NaN	NaN
2006-01-15	1486.730286	NaN	NaN	NaN

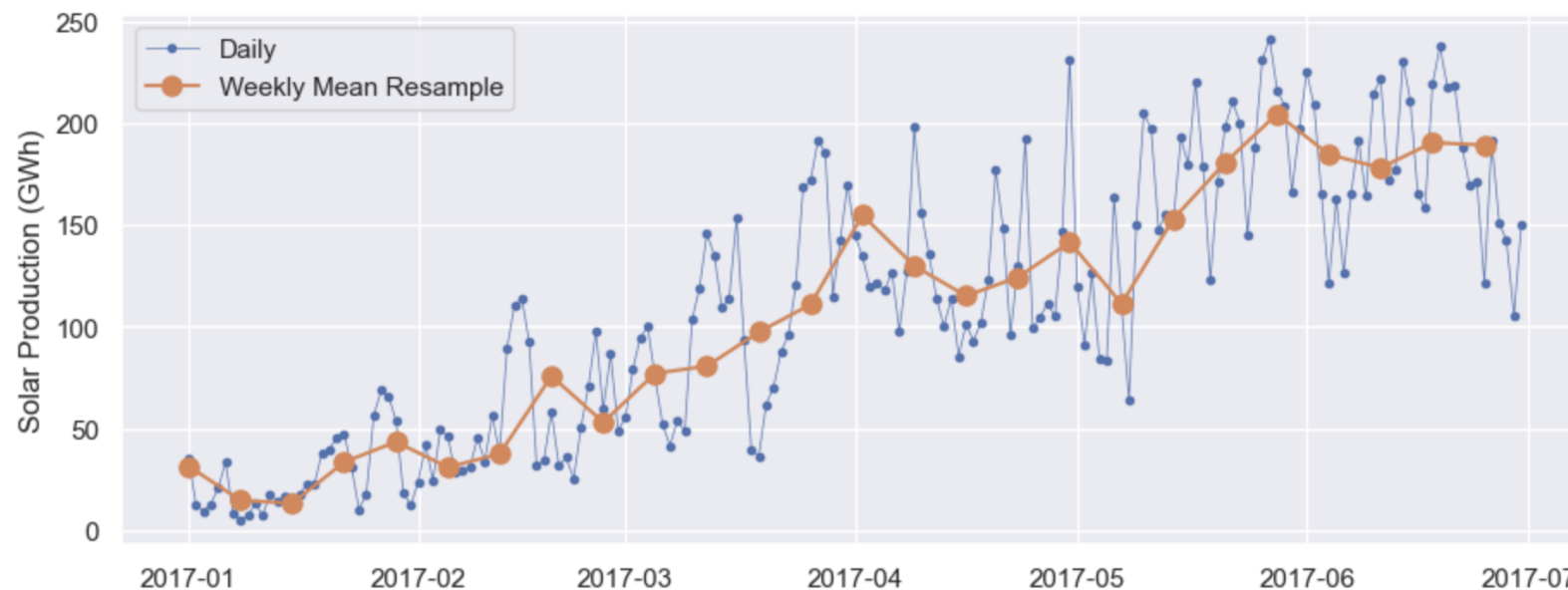
Open Power System Data



Resampling

```
1 # Start and end of the date range to extract
2 start, end = '2017-01', '2017-06'
3 # Plot daily and weekly resampled time series together
4 fig, ax = plt.subplots()
5 ax.plot(opsd_daily.loc[start:end, 'Solar'],
6 marker='.', linestyle='-', linewidth=0.5, label='Daily')
7 ax.plot(opsd_weekly_mean.loc[start:end, 'Solar'],
8 marker='o', markersize=8, linestyle='-', label='Weekly Mean Resample')
9 ax.set_ylabel('Solar Production (GWh)')
10 ax.legend()
11 plt.show()
```

✓ 0.1s

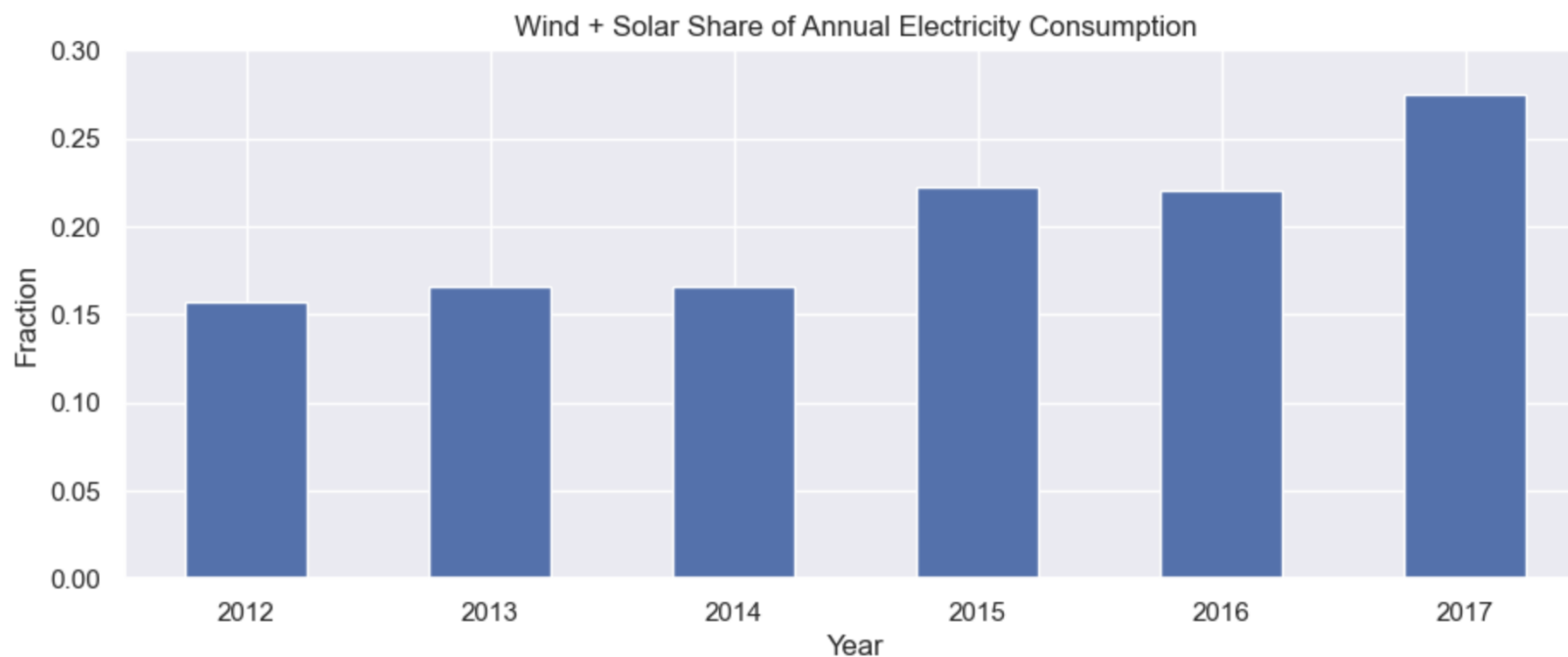


Open Power System Data



Resampling

	Consumption	Wind	Solar	Wind+Solar	Wind+Solar/Consumption
Year					
2015	505264.56300	77468.994	34907.138	112376.132	0.222410
2016	505927.35400	77008.126	34562.824	111570.950	0.220528
2017	504736.36939	102667.365	35882.643	138550.008	0.274500



Open Power System Data



Rolling Windows

```
1 # Compute the centered 7-day rolling mean
2 opsd_7d = opsd_daily[data_columns].rolling(7, center=True).mean()
3 opsd_7d.head(8)
```

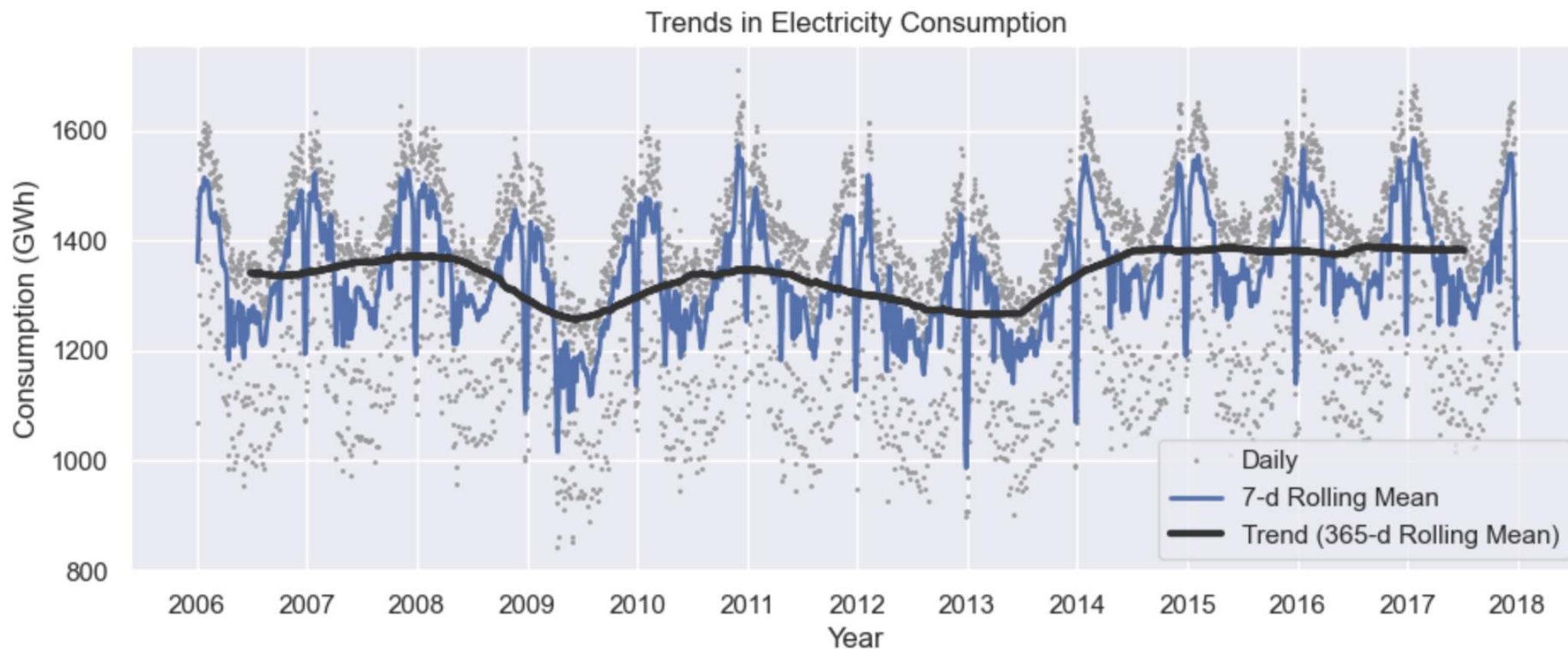
✓ 0.0s

	Consumption	Wind	Solar	Wind+Solar
Date				
2006-01-01	NaN	NaN	NaN	NaN
2006-01-02	NaN	NaN	NaN	NaN
2006-01-03	NaN	NaN	NaN	NaN
2006-01-04	1361.471429	NaN	NaN	NaN
2006-01-05	1381.300143	NaN	NaN	NaN
2006-01-06	1402.557571	NaN	NaN	NaN
2006-01-07	1421.754429	NaN	NaN	NaN
2006-01-08	1438.891429	NaN	NaN	NaN

Open Power System Data



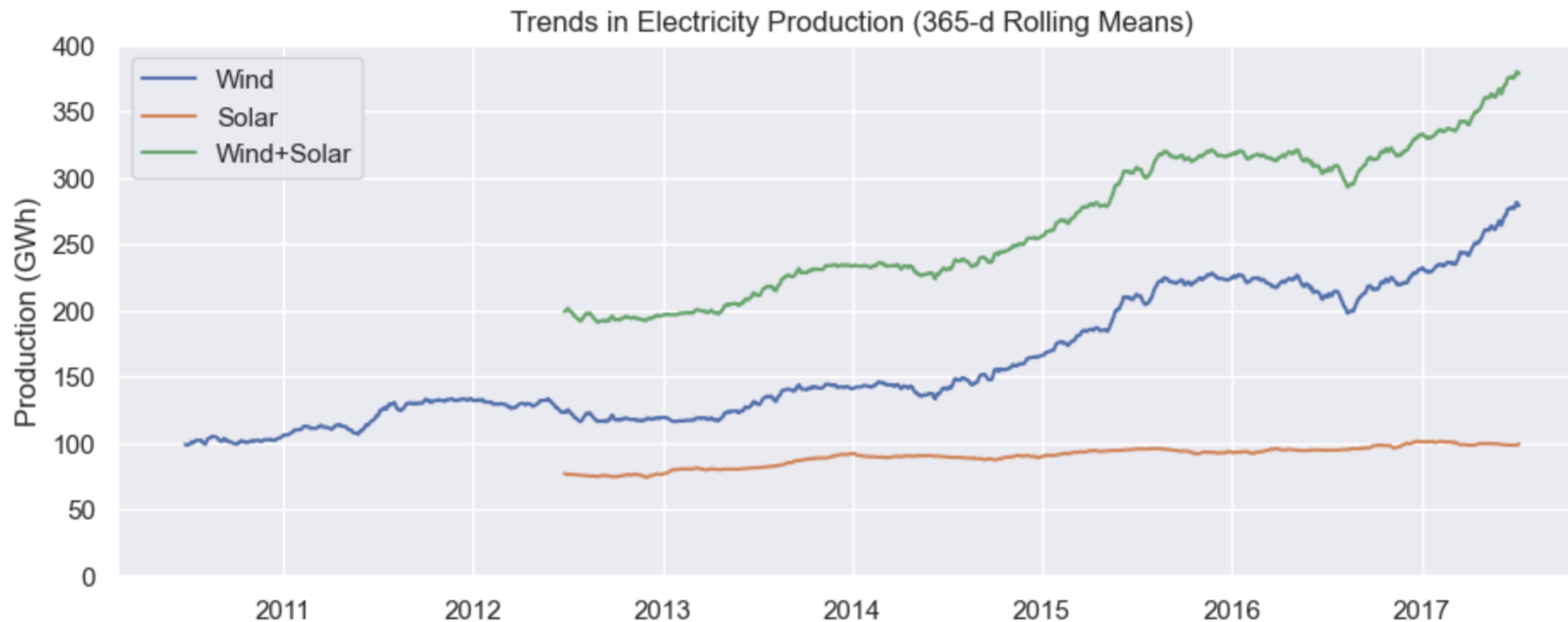
Trends



Open Power System Data



Trends



QUIZ TIME

Outline

SECTION 1

Introduction

SECTION 2

Applications

SECTION 3

Polars



Polars



Introduction Polars

Polars: A blazingly fast DataFrame library completely written in Rust, using the Apache Arrow memory model.





Why Polars

4525519 10.1021/jp030328p

4525520 10.1080/09500340802082418

4525521 10.1055/s-2002-36710

4525522 10.1001/jama.1920.02620510043027

4525523 10.7326/0003-4819-128-2-199801150-00014

4525524 rows x 5 columns

```
1 %%time
2 # pandas
3 first_df_pd = pd.read_csv(
4 |     csv_filepaths[0]
5 )
```

CPU times: user 33.1 s, sys: 4.21 s, total: 37.3 s
Wall time: 40 s

```
1 %%time
2 # polars
3 first_df_pl = pl.read_csv(
4 |     csv_filepaths[0]
5 )
```

CPU times: user 8.23 s, sys: 3.96 s, total: 12.2 s
Wall time: 12.7 s



Introduction

```
1 import polars as pl
2
3 dataset_path = 'Housing.csv'
4 # Read data from .csv file
5 data = pl.read_csv(dataset_path)
```

```
1 data
```

shape: (545, 13)

price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement
i64	i64	i64	i64	i64	str	str	str
13300000	7420	4	2	3	"yes"	"no"	"no"
12250000	8960	4	4	4	"yes"	"no"	"no"
12250000	9960	3	2	2	"yes"	"no"	"yes"
12215000	7500	4	2	2	"yes"	"no"	"yes"
11410000	7420	4	1	2	"yes"	"yes"	"yes"



Selection

```
1 price = data['price']  
2 price
```

shape: (545,)

price

i64

13300000

12250000

12250000

```
1 price = data.select('price')  
2 price
```

shape: (545, 1)

price

i64

13300000

12250000

12250000



Data Slicing

```
1 data.slice(  
2 | 10, 5  
3 )
```

shape: (5, 13)

price	area	bedrooms	bathrooms	stories
i64	i64	i64	i64	i64
9800000	13200	3	1	2
9681000	6000	4	3	2
9310000	6550	4	2	2
9240000	3500	4	2	2
9240000	7800	3	2	2

```
1 data.slice(  
2 | 20, 2  
3 ).select(  
4 | ['price', 'area', 'bedrooms']  
5 )
```

shape: (2, 3)

price	area	bedrooms
i64	i64	i64
8750000	4320	3
8680000	7155	3



Data Selection – Conditional Filtering

```
1 data.filter(  
2     (data['guestroom'] == 'yes') &  
3     (data['basement'] == 'no')  
4 )[['price', 'area', 'bedrooms', 'stories']]
```

shape: (26, 4)

price	area	bedrooms	stories
i64	i64	i64	i64
9800000	5750	3	4
8890000	4600	3	2
7962500	6000	3	4



GroupBy Operations

```
1 data.group_by('price').agg(pl.col('area').mean()).head()
```

shape: (5, 2)

price	area
i64	f64
5652500	6660.0
4403000	4226.666667
6230000	6050.0
8120000	6840.0
5075000	5720.0



Sorting Operation

```
1 data.group_by('price').agg(pl.col('area')  
2 | .mean()).sort(['area'], descending=True).head()
```

shape: (5, 2)

price	area
i64	f64
5943000	15600.0
10150000	12390.0
7343000	11440.0
7000000	11175.0
6930000	11040.0



Check Missing Values

```
1 data.null_count()
```

shape: (1, 13)

price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement
u32	u32	u32	u32	u32	u32	u32	u32
0	0	0	0	0	0	0	0



Dropping Columns

```
1 data.drop(['furnishingstatus']).head()
```

shape: (5, 12)

price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement
i64	i64	i64	i64	i64	str	str	str
13300000	7420	4	2	3	"yes"	"no"	"no"
12250000	8960	4	4	4	"yes"	"no"	"no"
12250000	9960	3	2	2	"yes"	"no"	"yes"
12215000	7500	4	2	2	"yes"	"no"	"yes"
11410000	7420	4	1	2	"yes"	"yes"	"yes"



Create New Column

```
1 data.with_columns(  
2     pl.lit(0).alias('temp_column')  
3 )
```

shape: (545, 14)

price	area	bedrooms	bathrooms	stories	mainroad	guestroom
i64	i64	i64	i64	i64	str	str
13300000	7420	4	2	3	"yes"	"no"
12250000	8960	4	4	4	"yes"	"no"
12250000	9960	3	2	2	"yes"	"no"
12215000	7500	4	2	2	"yes"	"no"



Applying Function

```
1 def binary_2_numeric(decision):
2     if decision == 'yes':
3         return 1
4     else:
5         return 0
6
7 applied_data = data.with_columns(
8     pl.col('mainroad').map_elements(binary_2_numeric),
9     pl.col('guestroom').map_elements(binary_2_numeric),
10    pl.col('basement').map_elements(binary_2_numeric),
11    pl.col('hotwaterheating').map_elements(binary_2_numeric),
12    pl.col('airconditioning').map_elements(binary_2_numeric),
13    pl.col('prefarea').map_elements(binary_2_numeric)
14 )
15 applied_data
```

shape: (545, 13)

price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement
i64	i64	i64	i64	i64	i64	i64	i64
13300000	7420	4	2	3	1	0	0
12250000	8960	4	4	4	1	0	0
12250000	9960	3	2	2	1	0	1

Summary

Introduction

- ❖ Pandas
- ❖ Pandas Data Structures
- ❖ Creating DataFrames
- ❖ Selection
- ❖ Dropping
- ❖ Sort & Rank
- ❖ Retrieving Series / DataFrame Information
- ❖ Applying Functions
- ❖ Data Alignment

Application / Exercise

- ❖ IMDB-Dataset
- ❖ Open Power System Germany Daily Dataset

Advanced

- ❖ Polars
- ❖ Compare: Pandas and Polars
- ❖ Functions: Selection, Slicing, ...



AI VIET NAM

@aivietnam.edu.vn

Thanks!

Any questions?